

Next up: the Professional exam. We're working on content for Claude Certified Architect (Professional). Suggestions, feature requests, anything really: [send feedback](#) and help shape it. Thanks, Walter

DISMISS

RESULTS

Exam Results

615 / 1000

NOT PASSED

17 of 28 correct (61%) · Pass mark: 720

Domain Breakdown

D1	Agentic Architecture & Orchestration	4/6	(67%)	+180pts
D2	Tool Design & MCP Integration	3/5	(60%)	+108pts
D3	Claude Code Configuration & Workflows	4/6	(67%)	+133pts
D4	Prompt Engineering & Structured Output	4/6	(67%)	+133pts
D5	Context Management & Reliability	2/5	(40%)	+60pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes $0.80 \times 0.27 \times 1000 = 216$ points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

HIDE DETAIL

TAKE ANOTHER EXAM

Question-by-Question Review

ALL 28

CORRECT 17

INCORRECT 5

SKIPPED 6

Q001 · D1 1.5 AGENT-SDK-HOOKS / POSTTOOLUSE-NORMALISATION q-1-5-002 ·

Customer Support Resolution Agent

CORRECT

A customer support agent integrates with three different backend systems via MCP tools. One system returns dates as Unix timestamps, another returns ISO 8601 strings, and the third returns dates in 'DD/MM/YYYY' format. The agent occasionally misinterprets dates, leading to incorrect order lookup results. What is the correct fix?

- C** Implement a PreToolUse hook that converts all dates to ISO 8601 before passing them to the tools

PreToolUse hooks intercept outgoing calls before execution, not incoming results. Date normalisation needs to happen on tool results after execution. The correct hook type is PostToolUse, not PreToolUse.

- D** Implement a PostToolUse hook that normalises all date formats to ISO 8601 before the model processes the results

PostToolUse hooks intercept tool results after execution but before the model processes them. Converting all dates to a consistent ISO 8601 format ensures the model receives clean, unambiguous data regardless of which backend system produced it. This is the canonical use case for PostToolUse hooks.

- A** Add instructions to the system prompt explaining the three date formats and how to interpret each one

Relying on the model to correctly interpret heterogeneous formats is unreliable. The model may confuse DD/MM/YYYY with MM/DD/YYYY or miscalculate Unix timestamps. Data normalisation should happen deterministically, not probabilistically.

- B** Standardise the backend APIs to all return the same date format

While ideal, modifying three separate backend systems is often impractical and outside the agent architect's control. The Agent SDK provides PostToolUse hooks precisely for normalising heterogeneous outputs without modifying upstream systems.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PostToolUse data normalisation)

Anthropic: Agent SDK Hooks ↗

Q02 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / GRACEFUL-DEGRADATION q-1-4-009 ·

Customer Support Resolution Agent

CORRECT

A customer support agent's loop terminates when cumulative token usage exceeds a budget. During a complex billing investigation the budget is exhausted mid-task, after data gathering but before resolution, with stop_reason still 'tool_use'. What architectural change addresses this?

C Remove the token budget entirely and rely solely on the iteration cap for safety

Removing the token budget eliminates a valuable cost control mechanism. The issue is not having a budget, but how the system handles budget exhaustion during active work.

D Switch to a cheaper model so each iteration uses fewer tokens, letting the agent fit more iterations into the same budget and finish the task.

A cheaper model may reduce the quality of the investigation itself. The architectural fix is graceful degradation when constraints are reached, not lowering the quality of work to fit within arbitrary limits.

A Double the token budget to ensure the agent always has enough tokens to complete complex investigations

Doubling the budget is an arbitrary fix that may still be insufficient for edge cases and wastes tokens on simple requests. The fundamental issue is that the budget terminates the loop even when stop_reason indicates the agent has more work to do.

B When the budget is nearly gone and stop_reason is still 'tool_use', summarise progress and escalate to a human.

When constraints prevent completion, the agent should degrade gracefully rather than terminating abruptly. Summarising progress and performing a structured handoff (customer ID, summary, root cause so far, recommended next steps) ensures no work is lost and the human agent can continue from where the AI left off.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Agentic loop lifecycle)

Lesson 1.4: Workflow Enforcement and Handoff (Structured handoff protocols)

Q03 · D3 3.3 HOOKS-AUTOMATION / SKILLS-CONTEXT-FORK q-3-3-007 ·

Technical Documentation Maintenance System

CORRECT

A documentation team creates a `/generate-api-docs` skill that reads source code files and produces Markdown API reference pages. The skill generates verbose output (200+ lines per endpoint) and should be available to every team member who clones the repository. How should the skill be configured?

- B** Create a `SKILL.md` in `.claude/skills/` with context: `fork`, so it is shared via git and isolates the verbose output

`.claude/skills/` is project-scoped and version-controlled, making it available to every developer who clones the repository. context: `fork` isolates the verbose 200+ line output per endpoint into a forked context, preventing it from consuming the main conversation's context budget.

- C** Add the documentation generation instructions to the root `CLAUDE.md` so they load automatically in every session

`CLAUDE.md` is for always-loaded standards, not for on-demand command definitions. Loading verbose generation instructions in every session wastes context tokens when the team is doing non-documentation work.

- A** Create a `SKILL.md` in `~/.claude/skills/` with context: `fork frontmatter`, and instruct each team member to copy it locally

`~/.claude/skills/` is user-scoped and not version-controlled. Requiring manual copying defeats the purpose of team-shared configuration and creates maintenance drift.

- D** Create a `SKILL.md` in `.claude/skills/` without any frontmatter, relying on the team to manually manage context overflow

Without context: `fork`, the verbose output (200+ lines per endpoint across many endpoints) would flood the main conversation context. The context: `fork frontmatter` is essential for isolating bulky output.

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (context: fork)

Claude Code: Skills ↗

Q04 · D2 2.4 MCP-SERVER-INTEGRATION / DESCRIPTIONS q-2-4-002 · Customer Support Resolution Agent

CORRECT

A customer support team configures an MCP server for their internal CRM system. The agent frequently ignores the CRM MCP tool and instead uses the built-in Grep tool to search local log files for customer information, producing incomplete results. The MCP tool's description reads: 'CRM tool'. What should the team do first?

- D** Expand the sparse 'CRM tool' description to detail its full customer records and structured output.

When an MCP tool's description is sparse, the agent defaults to familiar built-in tools like Grep. Enhancing the description to explain its full customer records, transaction history, account status, and structured output makes the agent prefer it for relevant queries. This is the recommended fix for MCP tool underutilisation.

- C** Move the MCP server configuration from `.mcp.json` to `~/.claude.json` so it takes higher priority.

Configuration scoping (project vs user level) does not affect tool selection priority. Both levels make tools available simultaneously. The issue is the tool description, not the configuration location.

- A** Remove the Grep tool from the agent's available tools so it cannot fall back to local file search.

Removing Grep cripples the agent for legitimate file search tasks. The problem is not that Grep exists but that the MCP tool's description does not communicate its superior capabilities for customer data.

- B** Add a system prompt instruction: 'Always use the CRM tool for customer queries. Never use Grep for customer data.'

Hard-coding tool preferences in the system prompt is brittle and does not scale. If new tools are added or the CRM tool is renamed, the instruction becomes stale or causes conflicts.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Enhancing MCP descriptions)

Q05 · D3 3.2 CLAUDE-MD-FILES / HOOKS q-3-2-008 · Technical Documentation Maintenance System

CORRECT

A documentation team wants to archive the full conversation transcript to a log file every time Claude Code runs `/compact`, so that context lost during compaction can be reviewed later. Which hook configuration achieves this?

- D** A `PostToolUse` hook on all tools that appends each tool result to a running log file, creating a continuous archive

Logging every tool result creates enormous log files and does not capture conversation context (reasoning, planning, user messages). The targeted `PreCompact` event captures the full transcript at the critical moment without per-tool noise.

- B** A `PreCompact` hook that writes the current transcript to a timestamped log file before `/compact` summarises the conversation

`PreCompact` is a first-class hook event in Claude Code that fires immediately before `/compact` (or auto-compaction) runs. The hook receives the full pre-compaction transcript path and can copy or process it before summarisation discards detail. This is the documented mechanism for exactly this use case.

- C** A `PreToolUse` hook configured on a built-in 'Compact' tool, matching tool name 'Compact'

There is no 'Compact' tool exposed to `PreToolUse`. Compaction is a Claude Code lifecycle operation, not a model-invoked tool, so it has its own dedicated `PreCompact` event rather than being hooked via `PreToolUse`.

- A** A `PostToolUse` hook on `Write` that snapshots the transcript whenever a file is written, on the assumption `/compact` will eventually run

`Write` and compaction are unrelated. `PostToolUse` on `Write` would fire on every file write whether or not `/compact` is about to run, producing noise rather than a transcript at the right moment. The dedicated `PreCompact` event is the correct hook.

WHERE THIS COMES FROM

Claude Code: Hooks ↗

Q06 · D4 4.4 VALIDATION-RETRY / DETECTED-PATTERNS q-4-4-004 ·

Content Moderation and Classification System

CORRECT

A moderation classification tool includes a `detected\_patterns` array where the model lists specific patterns it identified (e.g. 'repeated slur targeting ethnicity'). Validation checks whether the detected patterns match the assigned category, and on mismatch retries with feedback like 'You classified this as spam but detected patterns of hate speech targeting ethnicity, please re-evaluate.' What is the primary benefit of the `detected\_patterns` field in this workflow?

- B** It provides an auditable evidence trail showing which specific content features drove each moderation decision

Auditability is a secondary benefit. The primary benefit in this validation/retry workflow is enabling the system to detect and feed back reasoning inconsistencies for self-correction.

- C** It lets validation detect reasoning inconsistencies and feed back targeted errors the model uses to self-correct.

The detected_patterns field externalises the model's reasoning into structured data that can be programmatically validated against the conclusion. When patterns and category are inconsistent, the specific error feedback ('you detected hate speech patterns but classified as spam') gives the model actionable information to self-correct. This is the core benefit in a retry-with-error-feedback workflow.

- D** It increases classification accuracy by forcing the model to identify specific patterns before assigning a category

While structured reasoning fields can improve accuracy, this framing misses the retry workflow context. The primary benefit is enabling programmatic validation of reasoning consistency and targeted feedback for self-correction.

- A** It lets the system auto-correct the category by overriding the model's classification with a rule-based pattern match

The system does not override the model's classification. It feeds back the inconsistency and asks the model to re-evaluate. The model may correct its category or its patterns — the point is self-correction, not rule-based override.

WHERE THIS COMES FROM

Lesson 4.3: Prompt Chaining and Validation-Retry (detected_patterns fields)

Lesson 4.3: Prompt Chaining and Validation-Retry (Self-correction flow)

Q07 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-001 · Structured Data Extraction Pipeline

CORRECT

Your extraction pipeline validates that line item amounts sum to the stated total. For Document A, the sum is £450 but the stated total is £500. For Document B, the 'department' field is missing entirely from the source text. Which retry strategy is correct?

- B** Retry Document A with the discrepancy error; flag Document B for human review since the information is absent from the source

Document A has a fixable discrepancy (the model may have missed a line item). Document B has genuinely absent information — retries are ineffective.

- D** Skip retries for both and flag them for human review

Document A's discrepancy is likely fixable with error feedback. Skipping the retry wastes the model's self-correction capability.

- A** Retry both: send the validation error for Document A and instruct the model to find the missing department for Document B

Document B's department is absent from the source. Retrying will not make information appear that does not exist.

- C** Retry both documents with the same prompt since extraction is non-deterministic

Non-determinism does not create information that is absent. Document A may benefit from retry; Document B will not.

WHERE THIS COMES FROM

Lesson 4.3: Prompt Chaining and Validation-Retry (Retry effectiveness boundary)

Q08 · D1 1.5 AGENT-SDK-HOOKS / PRETOOLUSE-ENFORCEMENT q-1-5-015 · Customer Support Resolution Agent

CORRECT

A customer support agent handles account deletion requests. The company policy requires

explicit human approval before any account is permanently deleted. Currently, the agent sometimes processes deletions autonomously when the customer is insistent. What approval mechanism should the architect implement?

- A** Add a system prompt instruction: 'Always get manager approval before processing account deletions'

Prompt instructions are probabilistic. The question states the agent 'sometimes processes deletions autonomously when the customer is insistent,' showing that prompts alone are insufficient for irreversible operations that require 100% human approval.

- C** Remove the delete_account tool from the agent entirely and require customers to call a separate phone line

This creates a poor customer experience and eliminates the agent's ability to facilitate the process. The correct approach is to keep the tool but gate it behind human approval, providing a seamless escalation flow.

- B** Implement a PreToolUse hook on the delete_account tool that pauses execution and routes the request to a human approval queue before proceeding

A PreToolUse hook on the delete_account tool intercepts the call before execution and enforces the human approval requirement deterministically. The deletion cannot proceed until a human explicitly approves it, regardless of how insistent the customer is.

- D** Implement a PostToolUse hook that reviews the deletion after it has been processed and reverts it if no approval was recorded

PostToolUse hooks run after execution. Account deletion may be irreversible or difficult to undo. Prevention via a PreToolUse hook is required, not post-execution rollback.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Lesson 1.4: Workflow Enforcement and Handoff (Human approval gates)

Q09 · D3 3.4 PERMISSIONS-SECURITY / PLAN-MODE q-3-4-002

CORRECT

A developer is migrating a monorepo from CommonJS to ES modules: 80+ files, multiple valid strategies, knock-on effects in build tooling, test configuration, and `package.json`. They start in

direct execution mode and begin converting files. Halfway through, the chosen import resolution strategy breaks the test runner. What went wrong?

- B** The developer should have provided more detailed upfront instructions to direct execution specifying the exact migration pattern for every file type

The core issue is not insufficient instructions — it is that the developer did not explore the codebase to understand dependencies before choosing a strategy. More detailed instructions would still have used the wrong strategy because the test runner conflict was not known upfront.

- C** The developer should have used the interview pattern to have Claude ask questions about the migration before starting

The interview pattern helps surface considerations the developer might miss, but the fundamental issue is choosing direct execution for a task that requires codebase exploration and strategy evaluation. Plan mode, not just questioning, was needed to map dependencies and test impacts.

- A** The developer should have used plan mode to explore the codebase and evaluate migration strategies before committing to an approach

An 80+ file migration with multiple valid strategies and cross-cutting concerns (build tooling, test configuration, package.json) is a textbook plan mode scenario. Plan mode would have revealed the test runner incompatibility before any files were changed. Starting with direct execution for a complex, multi-approach task risks costly rework.

- D** The developer should have written a complete test suite first and iterated by sharing test failures

Test-driven iteration is an iterative refinement technique (task statement 3.5), not a substitute for upfront planning. The root cause is skipping plan mode for a complex task, not the absence of tests. The existing test runner itself was the source of the incompatibility.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode)

Lesson 3.4: Plan Mode vs Direct Execution (Recognising complexity)

A customer support agent has two tools: `get_customer_profile` (returns name, email, preferences) and `get_customer_orders` (returns order history with statuses). Users frequently ask 'Tell me about customer X', and the agent calls both tools sequentially, doubling response time. 90% of 'tell me about' queries only need the profile. What is the best approach?

- A** Consolidate both tools into a single `get_customer_info` tool that returns all customer data in one call.

Consolidation returns order data on every call, wasting resources for the 90% of queries that only need profile data. It also increases the response payload size unnecessarily.

- B** Refine the descriptions so `get_customer_profile` is the default and `get_customer_orders` is order-specific.

Refining tool descriptions with clear defaults is the lowest-effort fix. `get_customer_profile` should be marked the default for general 'tell me about' queries, directing the agent to call only the profile tool unless order details are explicitly requested. (correct answer)

- D** Set `tool_choice` to forced selection of `get_customer_profile` for all turns to prevent unnecessary order lookups.

Forced selection prevents the agent from ever calling `get_customer_orders`, even when users specifically ask about orders. This solves the over-calling problem by crippling legitimate functionality.

- C** Add a system prompt instruction: 'When a user says tell me about, only call `get_customer_profile`.'

Hard-coding keyword-to-tool mappings in the system prompt is brittle. It does not generalise to other phrasings like 'What do you know about customer X?' and will break if tool names change. (your answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Tool descriptions)

Q11 · D2 2.2 STRUCTURED-ERROR-RESPONSES / ACCESS-VS-EMPTY q-2-2-001 ·

Customer Support Resolution Agent

CORRECT

A tool returns an empty array after a customer lookup. The agent retries 3 times, then escalates to a human agent. Analysis shows the customer's account simply does not exist. What is the root cause of this wasted effort?

- C** The escalation threshold is too aggressive — the agent should exhaust more retries before involving a human.

The problem is not the escalation threshold. The problem is that the agent retries at all. A valid empty result requires no retry and no escalation.

- D** The system prompt should instruct the agent not to retry customer lookups.

Hard-coding retry rules per tool in the system prompt is brittle. The proper fix is structured error metadata that tells the agent whether the result is retryable.

- B** The tool does not distinguish between access failures and valid empty results, so the agent treats 'no matches' as a retrievable failure.

The tool successfully queried the data source and found no matches. This is a valid empty result, not an access failure. The agent's recovery logic cannot distinguish the two, causing it to retry when it should accept the result.

- A** The retry limit is too low — increasing to 5 retries would resolve the issue.

More retries make the problem worse. The tool succeeded — it found no matching customer. Retrying a successful query with no matches will never produce different results.

WHERE THIS COMES FROM

Lesson 2.2: Structured Error Responses (Access failure vs empty result)

Q12 · D4 4.4 VALIDATION-RETRY / RETRY-BOUNDARY q-4-4-003 ·

Content Moderation and Classification System

CORRECT

The moderation system validates that every classification includes a 'reasoning' field explaining the decision. When validation fails (empty reasoning), the system retries. For most posts, the retry succeeds after feeding back the error 'reasoning field was empty — provide a brief justification for the classification.' However, for posts in unfamiliar languages, retries consistently fail. What should the system do?

- C** Add the unfamiliar language to the system prompt as a supported language to encourage the model to try harder

Listing an unsupported language in the prompt does not give the model language understanding. This may actually worsen the problem by encouraging fabricated analysis of content the model cannot read.

- D** Remove the reasoning validation requirement for posts in unfamiliar languages

Removing the validation requirement means these posts would receive classifications without any justification, which undermines auditability. The correct approach is human review, not lower standards.

- B** Retry format errors on analysable posts; route capability gaps like unfamiliar languages to human review.

The retry boundary distinguishes between fixable format issues and unfixable capability gaps. Empty reasoning on analysable posts is a format error that retries with error feedback can fix. Unfamiliar languages represent absent capability — no amount of retrying will create language understanding. Route these to human review.

- A** Increase the retry count from 1 to 5 for unfamiliar language posts since the model may succeed with more attempts

More retries are ineffective when the model lacks the capability to analyse content in that language. The failure is not a transient format issue — the model genuinely cannot provide reasoning for content it cannot understand.

WHERE THIS COMES FROM

Lesson 4.3: Prompt Chaining and Validation-Retry (Retry boundary)

Q13 · D5 5.5 MONITORING-OBSERVABILITY / AGGREGATE-METRICS-TRAP q-5-5-001 ·

Structured Data Extraction Pipeline

CORRECT

A structured data extraction system achieves 97% overall accuracy across all document types. The team proposes automating all extractions where model confidence exceeds 95% to reduce human review costs. What is the critical risk in this approach?

- D** Automated extractions should always have human review regardless of confidence, making the proposal fundamentally flawed

Automation is valid when properly validated — the issue is doing it based on uncalibrated aggregate metrics, not the concept of automation itself.

- C** The model will become overconfident over time as it processes more documents, requiring regular retraining

LLMs do not train during inference. The issue is existing calibration gaps, not drift.

- A** The 95% confidence threshold is too low and should be raised to 99% for automation

The threshold value is not the core issue — the problem is that aggregate metrics hide per-type performance disparities.

- B** Aggregate accuracy can mask poor per-type performance, and confidence scores need calibration before use.

97% overall can hide 40% error rates on specific document types. Without stratified validation and confidence calibration, automation will silently fail on certain inputs.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Aggregate metrics trap)

Lesson 5.5: Human Review and Confidence Calibration (Confidence calibration)

Q14 · D2 2.5 BUILT-IN-TOOLS / GREP-THEN-EDIT q-2-5-006 ·

Technical Documentation Maintenance System

CORRECT

A documentation team needs to update all references to a renamed API endpoint across their 500,000-line codebase. They know the old endpoint name (POST /api/v1/users/create) but not which files reference it. After locating the files, they need to replace the old endpoint with the new one (POST /api/v2/users). What is the correct tool sequence?

- B** Use Read on the API router file to find all endpoint definitions, then manually trace each reference.

This approach only finds where the endpoint is defined, not where it is referenced. It also requires knowing which file contains the router definition, and manual tracing is unreliable across a large codebase.

- A** Use Glob for '**/*.md' to find all documentation files, then Read each one to check for the old endpoint, then use Edit to update matches.

Glob finds files by path pattern, but the endpoint reference could appear in any file type, not just Markdown. Reading every documentation file to search for the string wastes context tokens on irrelevant files.

- C** Grep for '/api/v1/users/create' to find every file with the old endpoint, then use Edit to replace it in each match.

Grep searches file contents across the entire codebase, locating all references regardless of file type or location. Edit then makes targeted replacements in each matched file. This is the minimal, precise approach.

- D** Use Bash with sed to perform a global find-and-replace across all files in a single command.

A global sed command bypasses the built-in tools designed for this task and risks unintended modifications without the ability to review each change. Edit provides targeted modifications with unique text matching for safety.

WHERE THIS COMES FROM

Lesson 2.5: Built-in Tools (Grep for content)

Lesson 2.5: Built-in Tools (Edit replacements)

Q15 · D5 5.1 CONTEXT-WINDOW-MANAGEMENT / PROGRESSIVE-SUMMARISATION q-5-1-009 ·

Technical Documentation Maintenance System

INCORRECT

A docs team processes a 500,000-line codebase by having Claude Code read files and generate docs, instructing it to 'summarise each module after documenting it, then discard the detailed source from context.' After processing 20 modules, the docs for module 21 reference class names from module 3 that were renamed in module 12. What caused this failure?

- B** The agent's context window overflowed, causing it to hallucinate class names from earlier modules

The team explicitly manages context by summarising and discarding source code, so the context window is not overflowing. The problem is the quality of information retained in summaries, not context overflow.

- D** The model confused similar class names because it processed too many modules in a single session

The question specifies the exact failure: module 21 references class names from module 3 that were renamed in module 12. This is not a confusion issue but a direct consequence of losing the rename detail during progressive summarisation. (your answer)

- C** Claude Code cached module 3's source code and served the stale version instead of the updated file on disk

Claude Code does not cache file contents between reads. Each Read tool call fetches the current file from disk. The issue is that summarised context retained old class names, not that file reads returned stale data.

- A** Progressive summarisation lost the rename detail: when module 12's summary replaced its source code, the rename was condensed to 'various refactoring changes', so the agent retained module 3's original class names from its earlier summary

Progressive summarisation is lossy. The summary of module 12 likely compressed the rename into a generic phrase, while module 3's summary still referenced the original class names. When the agent documented module 21, it found conflicting names across summaries and defaulted to the earlier, more prominent reference. Critical cross-module changes like renames must be tracked in a persistent facts block outside summarised context. (correct answer)

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Progressive summarisation)

Lesson 5.1: Context Window Management (Persistent facts)

Q16 · D3 3.6 CICD-INTEGRATION / STRUCTURED-VALIDATION q-3-6-007 ·

Technical Documentation Maintenance System

INCORRECT

A CI pipeline validates that generated API docs match the codebase. It runs ``claude -p 'Verify that all public API endpoints in src/api/ have corresponding documentation in docs/api/'`` with `--`

output-format json`. The check consistently reports 100% coverage, yet manual audits regularly find undocumented endpoints. What is the most likely cause?

- B** Claude Code is doing a shallow pattern match, not a deep semantic comparison, and the prompt lacks validation criteria

Without explicit validation criteria (e.g., 'for each exported function in src/api/, check that docs/api/ contains a section with the function name, parameters, and return type'), the model may perform a shallow check. Adding structured criteria, expected output format, and examples of what constitutes a gap produces reliable verification. **(correct answer)**

- D** The CI pipeline needs a separate review step where a second Claude Code instance verifies the first instance's findings

Adding a second instance to review the first adds latency and cost without addressing the root cause. If the first instance's prompt lacks clear validation criteria, the second instance reviewing the same vague output will draw the same flawed conclusions. **(your answer)**

- A** The --output-format json flag corrupts the analysis results, causing false positives

--output-format json controls the output structure, not the analysis logic. It formats results as JSON for downstream parsing but does not affect the accuracy of the verification itself.

- C** The -p flag prevents Claude Code from reading files, so it generates a plausible-sounding report without actual file access

The -p flag runs Claude Code in non-interactive (pipe) mode. It does not restrict file access — Claude Code retains full access to its built-in tools (Read, Grep, Glob) for reading files.

WHERE THIS COMES FROM

Lesson 3.6: CI/CD Integration (Structured output and validation)

Lesson 3.6: CI/CD Integration (CLAUDE.md for CI context)

Q17 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / AGENT-TOOL q-1-3-003 · Customer Support Resolution Agent

CORRECT

A coordinator agent should spawn specialist subagents for billing disputes and technical issues. Its system prompt describes the delegation workflow correctly, yet when it tries to invoke a subagent nothing happens: no subagent is created and no related error is thrown. What is the

most likely cause?

- C** The coordinator needs to pass the full conversation history to the subagent for it to initialise properly

Subagents do not inherit the coordinator's conversation history — context must be passed explicitly. However, this is not the cause of the failure to spawn. The fundamental issue is that the coordinator lacks the ability to invoke the Task tool.

- D** Subagents must be registered in a central agent registry that the coordinator loads at startup, and until they are, no Task call can resolve them.

There is no requirement for a subagent registry. Subagents are defined via an AgentDefinition and spawned through the Task tool. The real issue is that the coordinator's allowedTools omits 'Task' (or 'Agent').

- A** The coordinator's allowedTools list does not include the Task tool (renamed Agent in current Claude Code), so it cannot spawn subagents.

The Task tool is the mechanism for spawning subagents. If 'Task' (or its current alias 'Agent') is not in the coordinator's allowedTools, it physically cannot create subagents regardless of what the system prompt instructs. This is a configuration issue, not a prompt issue.

- B** The subagent's AgentDefinition is missing a description field, preventing it from being invoked

A missing AgentDefinition description would cause issues with the subagent's behaviour, but the problem here is that the coordinator cannot invoke the spawning mechanism at all. The issue is upstream of any subagent definition.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Task tool)

[Anthropic: Claude Agent SDK Overview](#) ↗

Q18 · D3 3.4 PERMISSIONS-SECURITY / PLAN-MODE q-3-4-008 ·

Technical Documentation Maintenance System

CORRECT

A documentation team must update architecture guides across 25 microservice directories after a platform migration. Each guide requires reading source, checking dependencies, and rewriting affected sections. Working in direct execution mode, a team member updates three services

before noticing the results are inconsistent: some guides still mention the old platform, others use inconsistent terminology for the new one. What went wrong and what is the correct approach?

- D** The team member should have used the Explore subagent to discover all affected files first, then made all changes in a single batch

Discovery alone does not solve the consistency problem. Without an explicit plan defining terminology and update patterns, batch execution would produce the same inconsistencies faster.

- A** The team member should have used plan mode to set a consistent update strategy and terminology before executing across all 25 services

A 25-service migration with consistency requirements is a multi-step, cross-file operation where planning is essential. Plan mode would produce an explicit strategy covering terminology, update order, and validation criteria before any changes are made, preventing the inconsistency that emerged from ad hoc execution.

- B** Direct execution was correct but should have been done in a single continuous session to maintain consistency through conversation context

A single session updating 25 services would exhaust the context window, causing later updates to lose awareness of earlier decisions. Context degradation would worsen the inconsistency problem, not solve it.

- C** The team member should have created a custom slash command that hardcodes the exact text replacements for each service

Architecture guides require contextual understanding of each service's code and dependencies — not mechanical text replacement. A slash command cannot adapt to the unique structure of each guide.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode for consistency)

Lesson 3.4: Plan Mode vs Direct Execution (Decision framework)

Q19 · D2 2.1 TOOL-SCHEMA-DESIGN / BOUNDARY-DESCRIPTIONS q-2-1-006 ·

Customer Support Resolution Agent

INCORRECT

A platform has `search\_knowledge\_base` ('Searches help articles') and `process\_action` ('Handles customer actions like refunds and plan changes'). The agent picks `search\_knowledge\_base` for 'cancel my subscription' requests. After adding 'subscription

cancellations' to `process\_action`'s description, it now picks `process\_action` for knowledge queries like 'how does cancellation work?'. What is the most effective solution?

- C** Consolidate search_knowledge_base and process_action into a single tool that determines the correct action based on user intent.

Consolidation pushes the routing problem inside the tool implementation and loses the LLM's natural language understanding. Keeping tools separate with clear boundary descriptions is simpler and more maintainable.

- A** Add boundary descriptions to both tools that separate executing a cancellation from learning how cancellation works.

Boundary descriptions resolve the ambiguity by intent. process_action should say to use it when the customer wants to execute a cancellation; search_knowledge_base should say to use it when the customer wants to learn about the process without taking action. Stating both boundaries prevents cross-routing in either direction. (correct answer)

- B** Remove the word 'cancellation' from both tool descriptions to eliminate the keyword conflict entirely.

Removing relevant keywords makes both tools less discoverable for cancellation-related queries. The solution is to differentiate when each tool handles cancellation, not to hide the concept from both.

- D** Add few-shot examples to the system prompt showing five cancellation scenarios with the correct tool for each.

Few-shot examples consume tokens and do not scale to the many variations of cancellation queries. Boundary descriptions in tool definitions are more efficient and generalise better. (your answer)

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (Boundary descriptions)

Lesson 2.1: Tool Interface Design (Misrouting)

Q20 · D5 5.5 MONITORING-OBSERVABILITY / STRATIFIED-SAMPLING q-5-5-002 ·

Structured Data Extraction Pipeline

INCORRECT

A legal document extraction pipeline has been running for three months with calibrated confidence thresholds. Extractions above 90% confidence are auto-approved without human

review, saving 60% of reviewer time. A new client submits contracts written in an unusual two-column legal format the system has not encountered before. The system reports 92% confidence on these extractions. What is the appropriate safeguard?

- D** Add a document format classifier that flags unknown formats for mandatory human review before any auto-approval

While not wrong in principle, this adds ML infrastructure complexity that stratified sampling already handles. A format classifier requires training data and maintenance, whereas ongoing stratified sampling detects novel error patterns without additional models. (your answer)

- A** Maintain stratified random sampling of high-confidence extractions across document types to catch the new format's error rate.

Stratified random sampling is designed precisely for this scenario: detecting novel error patterns in high-confidence extractions that slip past threshold-based routing. By sampling across document types, the new format's errors would surface during routine verification even though the confidence scores appear acceptable. (correct answer)

- C** Raise the confidence threshold from 90% to 98% for all document types to account for novel formats

Raising the threshold globally penalises well-understood document types that are genuinely accurate at 90%+. This wastes reviewer capacity on documents that do not need review, while still not guaranteeing detection of format-specific issues.

- B** Trust the calibrated confidence threshold since it was validated against labelled data and the system reports 92% confidence, which exceeds the 90% auto-approval threshold

Confidence calibration is only valid for document types represented in the calibration data. A novel format the system has never encountered can produce miscalibrated confidence scores — the model may be systematically overconfident because it does not recognise its own unfamiliarity with the format.

WHERE THIS COMES FROM

Lesson 5.5: Human Review and Confidence Calibration (Stratified sampling)

A customer support agent handles a multi-issue session. After several turns, the agent refers to 'your recent refund request' instead of the specific \$247.83 refund for order #8891. The conversation history is being summarised between turns to manage context length. What is the most effective fix?

- C** Instruct the model to preserve every amount, date, and order number verbatim whenever it summarises earlier turns.

Prompt-based instructions for summarisation are unreliable — the model will still compress details probabilistically.

- D** Store the full conversation in an external database and retrieve relevant turns on demand

This adds infrastructure complexity without addressing the core issue of which facts must persist in every prompt.

- B** Extract transactional facts into a persistent case facts block outside the summarised history.

This directly addresses the progressive summarisation trap by ensuring critical numerical and transactional data is never condensed.

- A** Increase the context window size to avoid summarisation entirely

This postpones the problem but does not solve it — eventually the context will fill, and summarisation will still destroy specifics.

WHERE THIS COMES FROM

Lesson 5.1: Context Window Management (Progressive summarisation)

Lesson 5.1: Context Window Management (Persistent facts)

Q22 · D4 4.3 STRUCTURED-OUTPUT / TOOL-CHOICE-FORCING q-4-3-002 · Structured Data Extraction Pipeline

CORRECT

Your legal document extraction pipeline uses `tool_use` with `tool_choice` set to 'auto'. The pipeline processes contracts and sometimes receives plain text analysis instead of the structured JSON extraction you need. The team suggests switching to prompt-based JSON with explicit formatting

instructions. What is the correct approach?

- D** Add stronger instructions in the system prompt telling the model to always use the extraction tool and never respond with plain text

Instructions alone cannot guarantee tool use when `tool_choice` is 'auto'. The API-level `tool_choice` parameter is the correct mechanism for enforcing structured output, not prompt instructions.

- A** Switch to prompt-based JSON as suggested, since the model clearly prefers text responses for these documents

Prompt-based JSON is less reliable than `tool_use` and can produce malformed output. The issue is `tool_choice` configuration, not the `tool_use` mechanism itself.

- C** Keep `tool_use` and set `tool_choice` to force the specific extraction tool by name, guaranteeing structured output on every request.

Forcing a specific tool with `tool_choice {type: 'tool', name: 'extract_contract'}` guarantees the model calls that exact tool every time, eliminating both text-only responses and wrong-tool selection. This is the most reliable approach for guaranteed structured output.

- B** Keep `tool_use` but switch `tool_choice` from 'auto' to 'any' to guarantee the model always returns a structured tool call

While 'any' guarantees a tool call, it lets the model choose which tool. If you have multiple tools defined, the model might call the wrong one. For a single extraction tool, this works, but the most precise solution is to force the specific extraction tool.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (tool_choice forcing)

[Anthropic: Tool Use Documentation](#) ↗

Q23 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-COVERAGE q-4-2-001 · Structured Data Extraction Pipeline

SKIPPED

Your extraction pipeline correctly identifies data in structured tables but returns empty fields when the same information appears in narrative paragraphs. Detailed instructions specify all required fields. What should you try first?

D Add a post-processing retry that re-extracts any empty fields

Retrying without better guidance will produce the same empty results. The model needs examples of correct extraction from narrative text.

B Add few-shot examples showing correct extraction from both structured tables and narrative paragraphs

Few-shot examples demonstrating correct handling of varied document structures directly address inconsistent extraction quality across formats. (correct answer)

C Add a pre-processing step to convert all narrative text into tables

This adds unnecessary complexity. The model should handle varied formats — few-shot examples teach it how.

A Increase the model's context window to process more of the document

The model already finds data in tables. The issue is not context size but inconsistent handling of different document structures.

WHERE THIS COMES FROM

Lesson 4.4: Few-Shot Prompting (Few-shot covering varied input)

Lesson 4.2: Structured Output with Tool Use (What tool_use does not enforce)

Q24 · D1 1.1 AGENTIC-LOOPS / ANTI-PATTERNS q-1-1-002 · Customer Support Resolution Agent **SKIPPED**

A customer support agent runs to its `max\_iterations=10` cap on every request, even simple ones. Logs show stop_reason returns 'end_turn' after the third iteration, but the loop ignores it and uses the iteration count as its primary termination signal. What architectural change resolves this?

D Parse the agent's response text after each iteration to detect phrases like 'task complete' and terminate early when found

Parsing natural language for completion signals is an anti-pattern. The model may use such phrases while still having tool calls queued, and may complete work without such phrases. The stop_reason field already provides a deterministic, unambiguous signal.

- A** Increase the iteration cap to 20 so the agent has more room to work on complex requests

Increasing the cap does not address the root cause. The agent already finishes simple tasks by iteration 3 (stop_reason is 'end_turn') but the loop ignores this signal and keeps running. A higher cap would waste even more iterations.

- B** Lower the iteration cap (for example to 3) so the loop is forced to terminate earlier on simple requests

This keeps the iteration count as the primary termination signal, which is the anti-pattern itself. Hard-coding a lower cap happens to fit this simple case but truncates genuinely complex requests that legitimately need more iterations, and it still ignores the stop_reason field that already reports completion. The fix is to make stop_reason the control signal, not to retune the cap.

- C** Make stop_reason the primary loop control; keep the iteration cap only as a safety backstop, not the main stop.

The stop_reason field is the authoritative signal for loop control. The loop should exit on 'end_turn' and continue on 'tool_use'. An iteration cap can remain as a safety net for edge cases, but must not be the primary stopping mechanism, which is the anti-pattern causing unnecessary iterations here. (correct answer)

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Anti-patterns)

Lesson 1.1: Agentic Loops (Agentic loop lifecycle)

Q25 · D4 4.3 STRUCTURED-OUTPUT / NULLABLE-FIELDS q-4-3-001 · Structured Data Extraction Pipeline

SKIPPED

Your extraction system uses tool_use with a strict JSON schema. All fields are marked as required. Testers report that the model invents plausible-looking dates and amounts when processing documents that lack this information. What is the best fix?

- D** Add a validation step that checks all values against the source document

Post-hoc validation is useful but does not address the root cause. Making fields optional prevents fabrication at the schema level.

B Switch from tool_use to prompt-based JSON extraction

This moves backwards in reliability. Prompt-based JSON introduces syntax errors without solving the fabrication problem.

A Add an instruction telling the model not to hallucinate

Vague instructions do not override the schema constraint. Required fields pressure the model to produce values.

C Make fields optional/nullable when source documents may not contain the information

Optional/nullable fields allow the model to return null instead of fabricating values. This directly prevents fabrication for missing information. (correct answer)

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Nullable / optional fields)

Q26 · D5 5.6 PRODUCTION-RELIABILITY / STRUCTURED-PROVENANCE q-5-6-008 ·

Technical Documentation Maintenance System

SKIPPED

A docs team has Claude Code generate architecture guides by synthesising source code, inline comments, commit messages, and ADRs. After several edits, the guides no longer indicate which statements came from which source. A developer later questions whether a specific architectural constraint in a guide is still valid. What approach preserves source attribution?

D Version-control the documentation and use git blame to trace each line back to the commit that created it

git blame traces authorship of documentation lines to commits, not to the source material that informed those lines. A commit message might say 'update architecture guide' without indicating that a specific statement came from ADR-047 or a comment in auth-service/src/middleware.ts.

C Keep all original source files unchanged in a reference directory so developers can manually trace any claim back to its origin

Preserving source files without explicit mappings forces developers to manually search through potentially hundreds of files to verify a single claim. This is not scalable and does not indicate which specific source informed which specific documentation statement.

- B** Maintain structured provenance metadata that maps each claim to its source file, line number, and retrieval date.

Structured provenance metadata survives editing because it is stored as data fields (e.g., JSON or YAML frontmatter) separate from the prose. When a developer questions a claim, they can trace it to the exact source file and line number. The retrieval date indicates whether the source was current when the documentation was generated. (correct answer)

- A** Instruct Claude Code to add footnotes to the generated documentation citing the original source for each statement

Prose footnotes are fragile — they are lost or degraded during subsequent editing passes, summarisation, or reformatting. The team has already experienced this problem through 'several iterations of editing.'

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Attribution preservation)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Structured claim-source mappings)

Q27 · D3 3.5 SLASH-COMMANDS-MCP / WORKTREE-PARALLEL q-3-5-008 ·

Technical Documentation Maintenance System

SKIPPED

A documentation team needs to simultaneously update API reference docs for three independent microservices after a breaking change. Each update requires reading source code, updating Markdown files, and validating links. A single Claude Code session would exhaust the context window trying to hold all three services' code simultaneously. What is the recommended approach?

- D** Split the documentation files into smaller chunks and process each chunk in a separate API call using the batch API

The batch API is for high-throughput, latency-tolerant workloads — not for interactive Claude Code sessions. Documentation updates require interactive exploration of source code and iterative editing, which the batch API does not support.

- B** Use git worktree for three branches, each with its own Claude Code session on one service, then merge the results

git worktree creates separate working directories on different branches, each with its own isolated Claude Code session. Each session has a full context budget dedicated to one service. The three updates run in parallel without interfering with each other, and results are merged via standard git workflows. (correct answer)

- C** Create a single skill with context: fork that processes all three services in parallel within one session

context: fork isolates a skill's output from the main conversation but does not create multiple parallel execution contexts. A single skill cannot simultaneously process three independent services in parallel.

- A** Process the three services sequentially in the same session, running /compact between each service to free context

Even with /compact, each service's context (source code, documentation, edits) is substantial. Sequential processing risks context degradation as compaction summaries lose detail from earlier services, and it is slower than parallel processing.

WHERE THIS COMES FROM

[Anthropic: Claude Code Documentation](#) ↗

[Git: Worktrees](#) ↗

Q28 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / PARALLEL-SPAWNING q-1-3-011 ·

Content Moderation and Classification System

SKIPPED

The coordinator receives a batch of 200 flagged posts to moderate. Currently it processes them sequentially, taking 45 minutes. Each post's moderation is independent — the decision on one post does not affect others. How should the coordinator handle this batch?

- C** Split into fixed batches of 20 posts and process each batch sequentially in a single prompt

Batching 20 posts into a single prompt still risks attention dilution across posts within each batch. And sequential batch processing still takes much longer than parallel individual processing.

- D** Increase the iteration cap on the agentic loop to allow the agent more time to process all 200 posts

The iteration cap controls how many tool calls the agent can make within a single task, not how many independent tasks it can process. The issue is parallelism, not loop duration.

- B** Delegate independent posts to parallel subagent instances, with the coordinator aggregating results as they complete

Since each post's moderation is independent, the coordinator should delegate them to parallel instances. This reduces total processing time from sequential (45 minutes) to roughly the time of the slowest individual moderation, and a failure on one post does not block others. (correct answer)

- A** Process all 200 posts in a single API call by concatenating them into one large prompt

Concatenating 200 posts into a single prompt causes attention dilution. Quality degrades for posts later in the sequence, and a single failure blocks the entire batch.

WHERE THIS COMES FROM

Lesson 1.3: Subagent Invocation and Context Passing (Parallel spawning)

SUPPORT

Found this useful?

The site is free and always will be. If the mock exam helped, you can buy me a coffee. Support goes to keeping the lights on.



BUY ME A COFFEE